

CHAPTER 8

ARRAYS AND STRINGS

Array:

An array is a collection of elements of the same data type stored in contiguous memory locations. It allows multiple values to be stored under a single name and accessed using an index.

Java arrays can hold both primitive types (like int, char, boolean, etc.) and objects (like String, Integer, etc.)

When we use arrays of primitive types, the elements are stored in contiguous locations. For non primitive types, references to items are stored at contiguous locations.

After creating an array, its size is fixed; we can not change it.

```
public class Geeks{
```

```

public static void main(String[] args){

    // Primitive array
    int[] arr = {10, 20, 30, 40};
    int n = arr.length;

    System.out.print("Primitive  Array
-> ");
    for (int i = 0; i < n; i++)
        System.out.print(arr[i] + " ");

    System.out.println();

    // Non-primitive array (String
objects)
    String[] names = {"Lakshit",
"Rahul", "Pankaj"};

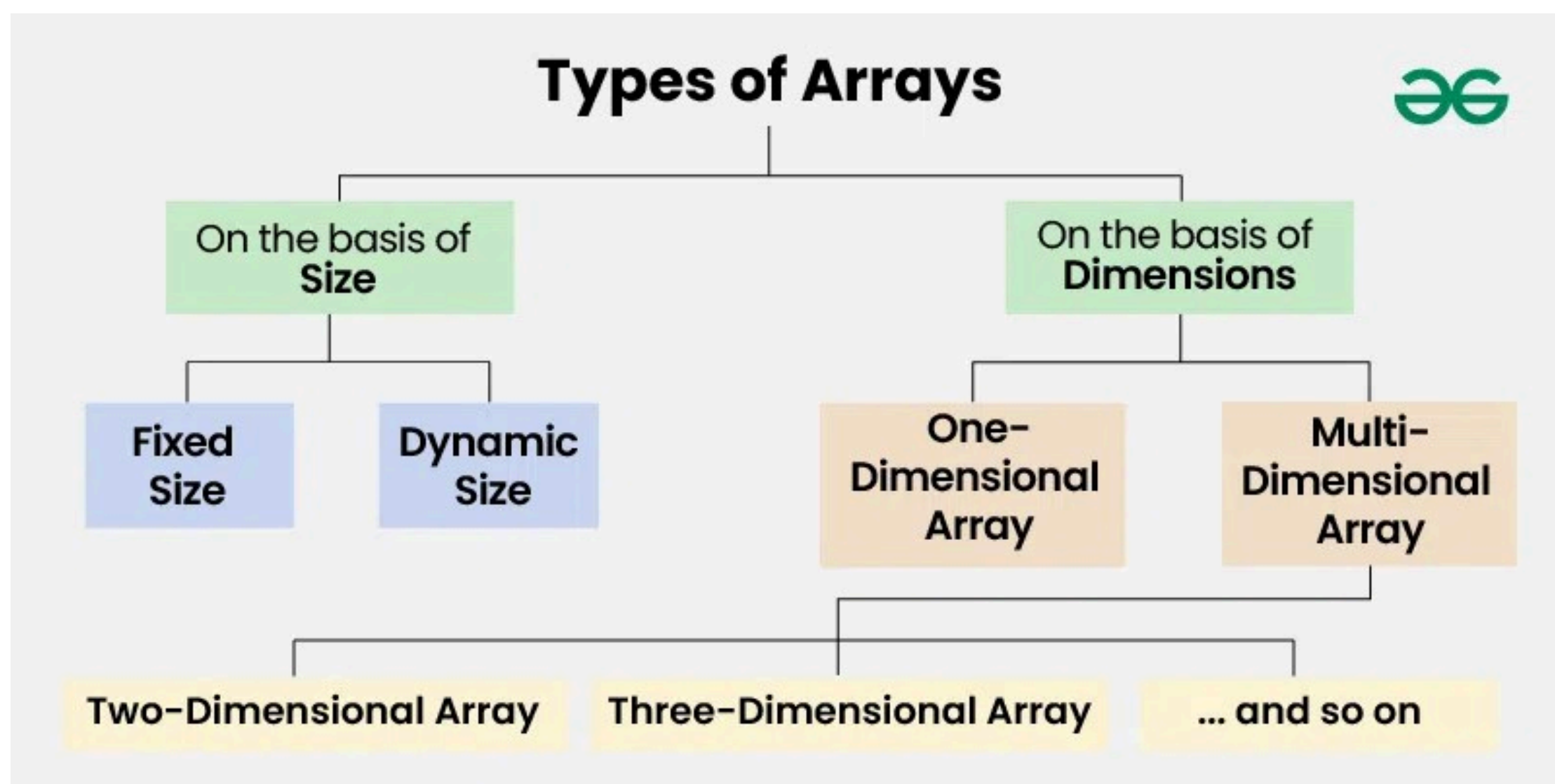
    System.out.print("Non-Primitive
Array -> ");
    for (int i = 0; i < names.length; i++)

```

```

        System.out.print(names[i] + "
    ");
    }
}

```



One-dimensional Array :




```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[5] = {1, 2, 3, 4, 5};
```

```
    // printing array elements
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

Two-dimensional (2D) array:

Col	→	0	1	2	
Row	↓	0	5	10	20
1		25	30	35	
2		1	3	4	

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}}; // 2
rows and 3 columns
```

```
    // printing array elements
```

```
    for (int i = 0; i < 2; i++)
```

```
    {
```

```
        for (int j = 0; j < 3; j++)
```

```
        {
```

```
            printf("%d ", arr[i][j]);
```

```
        }  
        printf("\n");  
    }  
  
    return 0;  
}
```

Jagged Array in Java:

Jagged array is a multidimensional array where each row can have a different number of columns. When we work with a jagged array, one thing to keep in mind is that the inner array can be of different lengths. It is like a 2D array, but each row can have a different number of elements.


```
class Geeks {  
    public static void main(String[] args){  
  
        // Declaring 2-D array with 2 rows  
        int arr[][] = new int[2][];  
  
        // Making the above array Jagged  
        arr[0] = new int[3];  
        arr[1] = new int[2];  
  
        // Initializing array  
        int count = 0;  
        for (int i = 0; i < arr.length; i++)  
            for (int j = 0; j < arr[i].length;  
j++)  
                arr[i][j] = count++;  
  
        for (int i = 0; i < arr.length; i++) {  
            for (int j = 0; j < arr[i].length;  
j++)  
                System.out.print(arr[i][j]
```

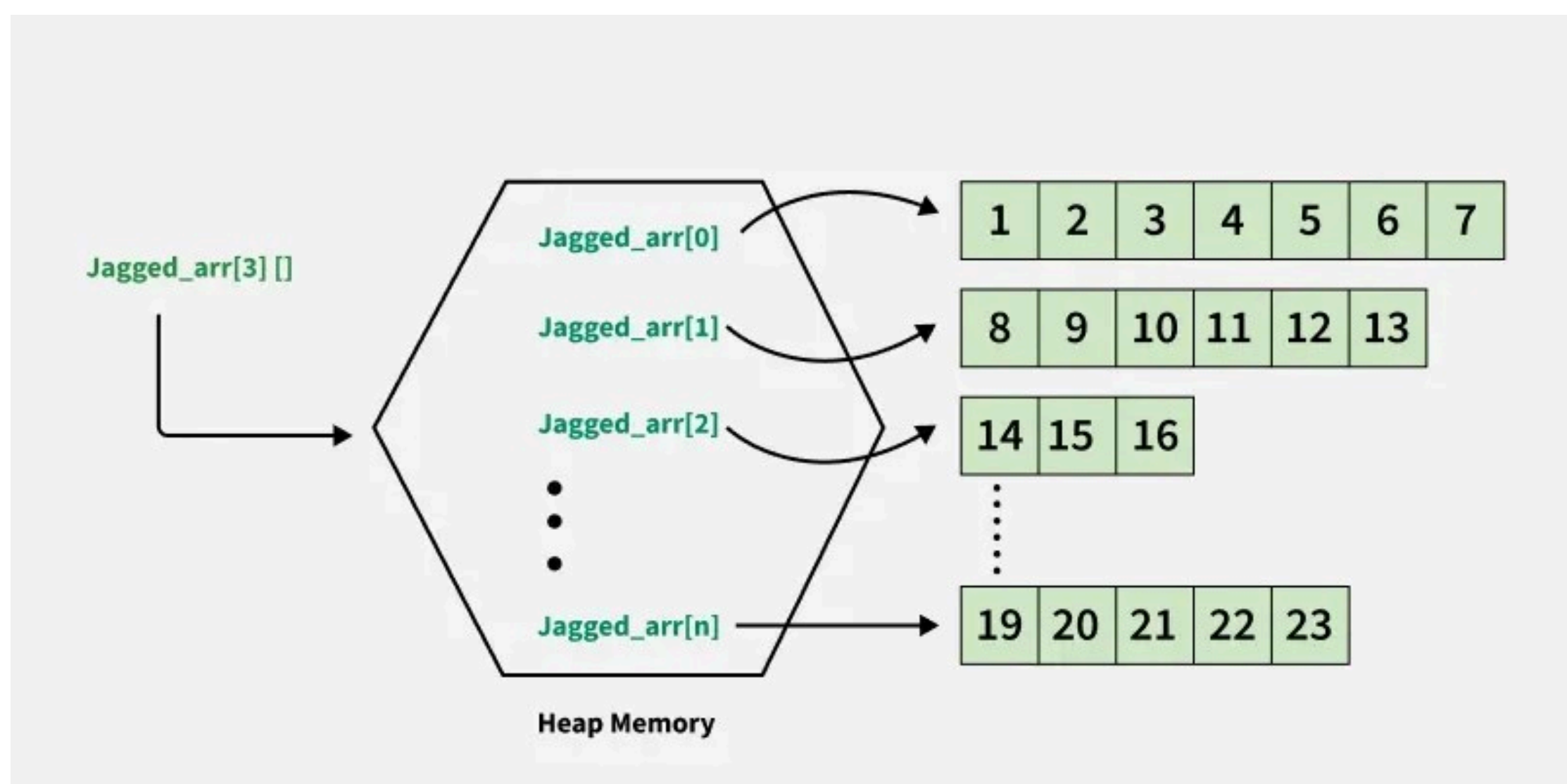
```
+ " ");
```

```
System.out.println();
```

```
}
```

```
}
```

```
}
```



Java Strings:

String in Java is an object used to store a sequence of characters enclosed in double quotes. It uses UTF-16 encoding and provides methods for handling text data.

Each character in a string is stored using 16-bit Unicode (UTF-16) encoding.

Strings are immutable, meaning their value cannot be changed after creation.

Java provides a rich API for manipulation, comparison, and concatenation of strings.

```
public class Geeks {  
  
    // Main Function  
    public static void main(String args[])  
    {  
  
        // creating Java string using a  
new keyword  
        String str = new String("Geeks");  
  
        System.out.println(str);  
    }  
}
```

String, String Buffer, String builder:

StringBuffer and StringBuilder are classes in Java used to create and modify mutable strings. Unlike the String class, their content can be changed without creating a new object. They are mainly used when frequent string modifications such as append, insert, or delete operations are required.

StringBuffer is thread-safe and synchronized, whereas StringBuilder is not synchronized and not thread-safe.

StringBuffer is slower in performance, whereas StringBuilder is faster and more efficient for single-threaded applications.

```
class Test {  
    public static void main(String[] args) {  
  
        StringBuffer sb1 = new
```

```
StringBuffer("Hello");
    StringBuffer sb2 = sb1;

    sb1.append(" World");

    if (sb1 == sb2)
        System.out.println("Same");
    else
        System.out.println("Not
Same");
    }
}
```

StringBuilder:

StringBuilder

StringBuilder is a mutable sequence of characters similar to StringBuffer , but it is not thread-safe. It is optimized for single-threaded environments where performance is critical.

Suitable for single-threaded applications.

Does not use synchronized methods.

Faster execution and better performance.

Syntax

```
StringBuilder sb = new  
StringBuilder("Hello");
```

```
class Test {  
    public static void main(String[] args) {  
  
        StringBuilder sb1 = new  
StringBuilder("Hello");  
        StringBuilder sb2 = sb1;  
  
        sb1.append(" World");  
  
        if (sb1 == sb2)  
            System.out.println("Same");  
        else  
            System.out.println("Not
```

```
Same");  
    }  
}
```